**Ivan Markeev**
Engineering Manager at Endava • CTO at FinPredict
Formerly at Google, Cisco, and Affirm

# Scaling Python Systems by Designing Team-Aware Architecture

Why your software boundaries are secretly team boundaries (and how to design both)

# The Scaling Myth: Coordination Tax Outpaces Runtime Performance

## THE PERFORMANCE TRAP

**When development slows down, organizations often look first at technology choices, frameworks, and runtime performance.**

But profiling the human cycle shows code execution is rarely the bottleneck.

Most organizations hit communication scaling limits long before they hit Python's execution limits.

## THE COORDINATION TAX

**Deployment Approvals**
Waiting days for reviews

**Shared Schema Merges**
Database lock friction

**Endless Status Syncs**
Aligning overlapping APIs

# The Hidden Dependency of Shared Data

The 15-Minute Task

## The Innocent Field

The checkout team needed to add a simple discount_code field to the database.

Writing the migration took only fifteen minutes of coding.

The Coordination Tax

## Multi-Team Gridlock

Shipping it required approval from Payments because they owned the transaction schema.

Analytics needed ETL updates, and Billing needed invoice modifications. Three teams had to align.

The Underlying Constraint

## Shared Boundary

The core problem was never the complexity of writing the migration.

**The database schema itself had become a massive, shared organizational boundary.**

# Is Your Architecture Restricting Your Team?

## TOPOLOGY VS. TOOLING

If developers spend more time negotiating dependencies than shipping software, your architecture is restricting them.

These aren't tooling failures—they are structural constraints.

## FOUR SOCIOTECHNICAL CODE SMELLS

**1. Deployment Gridlock**
Pushing a feature requires coordinated releases of 3+ independent services.

**2. Shared DB Pollution**
Direct cross-domain SQL joins crossing business boundaries.

**3. Sync Meeting Explosion**
Teams spending more hours in alignment than in editors.

**4. 'Works on My Machine' Defense**
Inability to run or test downstream services locally.

# Architecture Metric: Team Cognitive Load

LANGUAGE & BASICS

## Intrinsic Load

The fundamental mechanics of coding (e.g. Python syntax, testing libraries, asyncio mechanics).

**Maximize training and hiring to make this second nature.**

ENVIRONMENTAL NOISE

## Extraneous Load

Infrastructure and operational noise (e.g. deploying to staging, writing YAML, configuring local Docker schemas).

**Minimize through platforms and automation.**

CORE BUSINESS VALUE

## Germane Load

The actual business logic and domain processing (e.g. calculating regional taxes, writing the payment adapter).

**Maximize available mental space for this.**

SCAN TO ACCESS PUBLICATION

**The evolution of Cognitive Load Theory and the measurement of its intrinsic, extraneous and germane loads: a review**

Giuliano Orru, Luca Longo

**Published: February 2019**

# The Sociotechnical Contract: Team-First Architecture

OUTCOME & FLOW ORIENTED

## Stream-Aligned Team

Responsible for a single continuous stream of work in a business domain end-to-end ('you build it, you run it').

**Cross-functional, full-stack, and full-lifecycle, shielding developers from cross-team dependency delays.**

DEVELOPER EXPERIENCE CENTERED

## Platform Team

Responsible for building internal developer platforms to minimize extraneous load on stream-aligned teams.

**Provides pre-built, self-service golden paths in an 'X-as-a-Service' consumption model.**

SCAN TO ACCESS

**Team Topologies & TVP website**

Learn about Team-First Architecture and the Thinnest Viable Platform.

**teamtopologies.com**

# Conway's Law: Code Mimics Communication Structures

### The Inevitable Mirror

Organizations design systems that mirror their communication channels. Siloed teams generate siloed software; high-friction groups generate high-friction delivery.

**If you divide engineers into Frontend, Backend, and DBA silos, you will inevitably build a tightly coupled three-tiered monolith.**

### The Inverse Conway Maneuver

**1. Design Teams First** Restructure team dynamics to match the target architecture.

**2. Decentralize Authority** Empower cross-functional stream teams to release autonomously.

**3. Formalize Team APIs** Enforce clean boundaries by declaring code-level team interfaces.

SCAN TO WATCH

**Conway's Law and Generative AI**

Evolving Your Organisation Design — Patrick Debois

youtube.com

# Bounded Contexts: Linguistic Boundaries Prevent Model Contradictions

MARKETING REPRESENTATION

## Catalog Context

A product is a customer-facing entity.

**Fields:** descriptions, images, localized campaigns, categories.

**Language:** Marketing & Sales.

PHYSICAL ASSET

## Inventory Context

A product is a warehouse box with physical metrics.

**Fields:** SKUs, weight, dimensions, warehouse shelf locations.

**Language:** Shipping & Logistics.

FINANCIAL TRANSACTION

## Billing Context

A product is a tax-carrying ledger item.

**Fields:** tax codes, prices, ledger credits, discount rules.

**Language:** Accounting & Treasury.

SCAN TO WATCH

**Domain-Driven Design: Bounded Contexts Explained!**

ByteMonk

youtube.com

# Bounded Contexts are Engineering Tools, Not Org Charts

THE 1:1 MAPPING MYTH

## The Corporate Org Chart Trap

Pragmatic software contexts should be drawn based on technical specialties, deployment cadences, and security requirements—not to mirror administrative lines of a corporate manager's org chart.

## Engineering for Evolvability

We leave corporate domains as the business sees them, and draw our own Bounded Contexts to serve our need for system evolvability.

SCAN TO ACCESS PUBLICATION

**Domain-Driven Design in Practice: A Large-Scale Empirical Characterisation of the Open-Source Ecosystem**

Ozan Özkan, Önder Babur, Mark van den Brand

**Published: July 2026**

# Sizing Services Around Team Capacity & Domain Volatility

### COGNITIVE DENSITY HEURISTICS

Avoid the nanoservices anti-pattern, where a team of five developers is forced to manage forty microservices, causing extreme dependency and testing chaos.

Rule of thumb: a single team should own no more than 1 complex subdomain, or 2-3 simple subdomains.

### THE COARSE-TO-FINE GRANULARITY SPECTRUM

**Macro-services** COARSE & STABLE

Broader, highly stable business domains (e.g. General Ledger). Minimal updates, low operational overhead, coarse boundaries.

**Miniservices** BALANCED

Balanced domain scope. Delivers deployment independence and clear boundaries without excessive distributed systems cost.

**Microservices** FINE & VOLATILE

Volatile, highly dynamic capabilities (e.g. Promotion Calculations) where fine-grained services support daily releases.

# Why 'Database-per-Service'

STAGE 1 — LOGICAL ISOLATION

## Separate ownership, shared infrastructure

- Each service owns its own database/schema boundary.
- Separate credentials enforce ownership.
- Cross-domain SQL joins are replaced by APIs or events.
- Teams can evolve their data model independently.
- Infrastructure remains shared to reduce operational overhead.

STAGE 2 — PHYSICAL INDEPENDENCE

## Separate infrastructure where it creates value

- Move services to dedicated database instances when needed.
- Isolate compute, memory, storage, and operational concerns.
- Allow teams to scale and operate independently.
- Prevent one service's workload from impacting another.

# Event-Driven State Replication Data on the Outside

THE SHARING PROBLEM

When databases are strictly isolated per service, how do teams share data without creating synchronous, coupling HTTP calls that fail under load?

### DOMAIN EVENTS

When a state changes, the owning service publishes an immutable event (e.g. `UserUpdated`) to a message broker.

### MATERIALIZED VIEWS

Downstream teams subscribe to events and construct read-optimized local caches.

### ORGANIZATIONAL AUTONOMY

If the Account service goes down, Billing can still process invoices using its local materialized view. No runtime dependencies.

READ THE FOUNDATIONAL PAPER

**"Data on the Outside vs. Data on the Inside"**

Scan the QR code to read Pat Helland's seminal 2005 paper on SOA boundaries and message immutability.

EVENT PUBLISHING WITH FASTAPI & KAFKA

```python
# Account Service (Producer)
@app.post("/users/")
async def create_user(user_data: dict):
    db.save(user_data)

    # Publish domain event for other teams
    await kafka.send(
        topic="user.events",
        value={"event": "UserCreated", "id": user_data['id']}
    )

    return user_data
```

# Defining the Team API: REST vs. gRPC Contracts

### REST: FASTAPI DOCUMENTATION-DRIVEN

```python
# The contract depends on manual vigilance

@app.post("/orders/")
def create_order(order_data: dict):
    # Raw untyped payload
    # Are fields safe? Is schema updated?
    db.insert(order_data)
    return {"status": "created"}
```

### GRPC: PROTOBUF TOOLING-DRIVEN

```protobuf
// Enforces contract compile-time checks

message OrderRequest {
    string order_id = 1;
    int64 customer_id = 2;
    double amount = 3;
}

service Order {
    rpc Create(OrderRequest) returns (Resp);
}
```

# Safe Contract Evolution with Buf and Pydantic

API SAFEGUARDS IN CI

**01 gRPC Schema Protection**

Integrate Buf into CI pipelines. Running `buf breaking --against` blocks commits that accidentally alter types or fields.

**02 REST Schema Versioning**

Map internal models to public versioned Pydantic DTOs (e.g. `UserResponseV1`). Never expose database entities directly.

VERSIONED DTO SCHEMA (FASTAPI / REST)

```python
from pydantic import BaseModel

# Private DB entity is NEVER exposed
class DbUser(Base):
    __tablename__ = 'users'
    id = Column(Integer)

# Public versioned API contracts
class UserResponseV1(BaseModel):
    id: int
    email: str

class UserResponseV2(BaseModel):
    id: int
    email: str
    display_name: str
```

# Enforcing Boundaries with Architecture Tests in CI

AUTOMATED POLICIES IN PYTHON

**01 Boundary Deficit**

Bypassing contract endpoints to import code directly from another team's package destroys boundary integrity and couples domains.

**02 Continuous Verification**

Enforce rules programmatically using `import-linter`. Any forbidden cross-domain import immediately breaks the build in the CI pipeline.

.IMPORT-LINTER.INI (CI PIPELINE RULE)

```
[import-linter]
root_package = myapp

[import-linter:contract:billing-isolation]
name = Billing Isolation
type = forbidden
source_modules =
  myapp.shipping
forbidden_modules =
  myapp.billing
```

CI PIPELINE CONSOLE OUTPUT

```
ImportLinterError: Forbidden import found:
myapp.shipping → myapp.billing.models
```

# The Cost of Decoupling: Cognitive Load

01

AUTONOMY HAS A HIDDEN PRICE

## The Boilerplate Drown

When stream-aligned teams are forced to build and manage their own infrastructure, they drown in boilerplate instead of writing business logic.

02

EXCEEDING CAPACITY

## The Velocity Drop

If a team spends 40% of their sprint fighting infrastructure, velocity drops. Team Topologies warns that exceeding cognitive capacity destroys morale and quality.

THE COGNITIVE OVERFLOW

- Kafka Producer/Consumer configuration
- Dockerfile optimization
- Kubernetes YAML manifests
- CI/CD pipeline debugging
- gRPC scaffolding

*# Exceeding cognitive capacity destroys morale and quality*

# Platform as a Product: Self-Service Tooling

THE ENABLING SOLUTION

## Don't build a ticket-driven DevOps silo. Build an internal product.

### THE PAVED ROAD

The Platform Team provides a standard Python chassis that automatically handles:

- OpenTelemetry / Logging
- Kafka publishing interfaces
- gRPC stubs & CI testing

***Adoption is driven by making developers' lives easier, not by mandates.***

# Tracing: Settle Cross-Team Disputes with Data

## OBSERVABILITY AS A TEAM INTERFACE

Replaces subjective finger-pointing ('checkout is slow' vs 'payments are fine') with objective execution traces.

Visually traces the exact path of a single transaction across network boundaries, exposing the exact culprit with zero ambiguity.

**OPENTELEMETRY DISTRIBUTED TRACE (HONEYCOMB / JAEGER)**

```
TraceID: c370a92b0f4 (Latency: 2.5s)

[API Gateway] ──────────────────── (2.5s)
[Checkout-Service] ─────────────── (2.4s)
[Payment-Service] ─ (0.1s)
[Recommendation-Service] ──────── (2.3s)
```

```
# VERDICT:
96% of checkout request latency is spent waiting
on Recommendation Service. Payments are healthy.
```

# Reducing Coordination, One Boundary at a Time

1. CI/CD ENFORCEMENT

## Write Architecture Tests

Install import-linter. Define one forbidden import rule.

**Time to first value: 30 minutes.**

2. OBSERVABILITY

## Start Tracing

Instrument one service boundary with OpenTelemetry.

**Time to first value: half a day.**

3. DECOUPLING

## Isolate One Shared Table

Find one shared table and create an ownership plan.

4. PLATFORM ROADS

## Build One Golden Path

Find one painful operational task and automate it for everyone.

# “Don’t add a meeting — add a structural boundary.”

Evolving software systems by respecting the cognitive limits of human teams.

**Ivan Markeev**
LinkedIn
*"Scaling Tech, Scaling Teams, Scaling Impact."*